

Atlas Inventory Workshop Trainer Notes

Introduction

Attendees should not read this before the workshop. This document describes the workshop setup, planted defects, and the trap in each exercise.

Before the session

```
mvn clean verify                # 15 tests, all green
python3 -m json.tool opencode.json > /dev/null && echo "config OK"
opencode agent list | head -1    # must not error
```

Two facts worth verifying on the machines in the room, because they have moved between opencode releases:

- **Confirm attendees' model.** It should be `gpt-4.1-mini` (change using `/models`). Exercise prompts are tuned for it.
- **Plan mode's edit permission.** `opencode agent list` → the `plan (primary)` block. As of 1.18.20 it resolves to `edit / *` → `deny` , with an allow only for `.opencode/plans/*.md` . opencode's published docs still describe it as `ask` . `syntax/02` documents the `deny` behavior. If the version on attendees machines differs, warn them rather than letting attendees hit a denied edit they were told to expect approval for.
- **`opencode.json` is loaded at all.** A syntax error there makes opencode refuse to start in the directory, with an error that reads like an install problem rather than a config one.

The two planted defects

This repository ships one defect and following the exercises inject a second. They are unrelated, don't let them merge in your head.

1. Missing check for quantity

The `validateForUpdate` function doesn't check `quantity` (flaw already in `main` branch). In contrast, `InventoryService.validateForCreate` rejects a negative quantity, but `validateForUpdate` never checks it. Flaw was planted in commit `ca4e093`. `PUT /api/items/{id}` with `quantity=-5` succeeds and stores a negative stock level. No test covers this bug.

This is **exercise 4's payload**. The exercise asks attendees to factor out the duplicated *reorder level* check. and an agent told to "remove the duplication" will often unify both methods, silently adding the quantity check to the update path. The full suite still passes. That is the lesson: a green suite is evidence about the tests you have, not about the change you made.

It is also the second deliberate ambiguity in **task 5.A**.

Do not fix it on `main`.

2. The summing up defect

`totalReorderShortage` (injected for exercise 3) is added by the trainer on an `exercise-03` branch, per [exercise_03_preparation.md](#) — use the setup script there rather than hand-editing, and see that file for how to get the branch onto every attendee's machine at once. Sums `reorderLevel - quantity` without clamping, so surplus items cancel out real shortages: `-37` instead of `8`.

CI (`.github/workflows/build.yml`) runs `mvn --batch-mode verify` on every push and **will go red on that branch by design**. Don't merge the branch.

Traps

Per-exercise traps

Exercise 1 — read-only investigation. The answer is in `app.js` (`renderSummary`), not in Java. Watch for agents that open `InventoryRepository.java` or the schema and start theorising about SQL. The edge case worth landing: a `storageLocation` with no `-` still yields a "zone" (the whole string).

Exercise 2 — small feature. Straightforward; the point is the plan-then-approve rhythm. Check they ran `mvn test -Dtest=InventoryItemTest` and not a full `verify`. `InventoryItemTest.java` is deliberately *not* shipped — creating it is part of the task.

Exercise 3 — defect. Two failure modes to watch for:

- Watch for the agent editing the test instead of the code. The agent edits the assertion to `assertEquals(-37, total)`. Exactly the failure mode attendees need to learn to catch in review. Green suite, bug intact. This is the single most valuable thing to catch in front of the room.
- If exercise 2 was done first, `InventoryItem.reorderShortage()` already exists, and `total += item.reorderShortage();` is a *better* answer than the `Math.max(0, ...)` one-liner in the exercise text. Accept it and say why. Use it to make the point that "the smallest fix" and "the best fix" aren't always the same line.

Exercise 4 — refactoring. See defect 1 above. If nobody triggers it, demonstrate it yourself: run the prompt with the quantity invariant removed from the constraint list and diff the two results.

Exercise 5.E — error message. This one has teeth, so know the answer before you're asked.

`InventoryServerTest#reportsDuplicatePartNumbersAsConflict` asserts the body contains the exact substring `part number already exists`. Most natural rewrites that inject the part number break it — but appending works:

```
"An inventory item with that part number already exists: " + partNumber
```

The task forbids touching the test on purpose. The skill is noticing the constraint *before* editing, not negotiating it away afterwards.

Exercise 5.D — description boundary. `validateTextFields` is private, so the test has to go through `create()` — which needs a repository.

`InventoryServiceTest.java` ships with that wiring (`@TempDir` + real SQLite file). Steer anyone building a harness from scratch toward it.

The in-memory database trap

`InventoryRepository` opens a **new connection per operation**. An in-memory SQLite database is therefore empty again by the time any query runs — `initialize()` creates the schema in one database and `findAll()` queries a different, empty one.

The `:memory:` special case that used to advertise support for this has been removed precisely so agents can't be lured into it. If an attendee's agent invents `new InventoryRepository(Path.of(":memory:"))` anyway, the failure is `no such table: inventory_items`. Point them at `InventoryServiceTest`'s `@TempDir` setup.

Timing

- Reference reading ~30 min, hands-on 2 h 15 min, plus debriefs.
- Exercises 1 and 3 are the best candidates to run as a live group demo.

- Exercise 5 is meant to be unaided.